

自然语言处理

王刚

统计与数据科学学院

2026 年春季学期

目录

第二章词汇分析

2.1 语言中的词汇

2.2 词语规范化

2.3 中文分词

2.4 词性标注

第二章词汇分析

引言

为什么词汇分析在自然语言处理中至关重要？

- ▶ 因为许多下游自然语言处理任务，如文本分类、情感分析、机器翻译和命名实体识别等，都依赖于对文本中单词的准确识别和解释。
- ▶ 词汇分析是 NLP 的底层支柱，直接影响后续任务的准确性和可靠性。没有高质量的词汇分析，复杂的语义理解、推理和生成任务将失去基础。通过掌握词汇分析技术，我们可以更好地理解文本的结构和意义，从而实现更加复杂和准确的自然语言处理应用。
- ▶ 随着深度学习的兴起，虽然部分任务（如分词）可通过端到端模型隐式学习，但显式的词汇分析仍对模型可解释性和低资源场景具有重要意义。

2.1 语言中的词汇

什么是词？

- ▶ 语言学的定义：词（word）是形式和意义相结合的单位，也是语言中能够独立运用的最小单位。
掌握一个词汇意味着知道其读音和语义：
例如：英文单词 cat 具有的语义是猫，读音为 /kæt/
- ▶ NLP 中的映射：算法将词视为离散符号，需通过数字化（如词向量）表征其形式与语义。自然语言处理算法中“词”通常也是基本单元。
- ▶ 词的处理也是自然语言处理中重要的底层任务，是句法分析、文本分类、语言模型等任务的基础。

语言中的词汇

词 (word) 通常是由语素 (Morpheme) 构成。

语素又称词素，是语言中意义的最小单元。

- ▶ 语素与词不同，语素不能够独立运用而词可以（如英语后缀“-s”表复数，需依附于名词）。
- ▶ 只包含一个语素的词语称为简单词 (Simple word)。
- ▶ 包含多个语素的词称为复杂词 (Complex word)。
例如：“电灯”，包含“电”和“灯”两个语素。
- ▶ 根据词在语言中的用途的不同，词还可以被划分为实义词和
功能词。
实义词包含事物、行为、属性和观念等概念。承载具体语义（名词、动词等）。
功能词则是指没有清楚词汇意义或与之有关的明显概念的
词。表达语法关系（介词、连词等）。

词的形态学

- ▶ 虽然单词的形式和意义之间的关系本质上是任意的，但是由于社会的约定俗成，词的形式具有服从于某种规则的内在结构。
- ▶ 研究单词的内部结构和其构成方式的学科称为形态学 (Morphology)，又称构词学。
- ▶ 研究词的内部结构与构成规则，揭示形式（拼写/语音）与意义（语义）之间的系统化关联。
- ▶ 关键问题：社会约定俗成的规则如何通过语素组合生成新词，例如：中文“电”+“灯”=“电灯”（复合词）
- ▶ 词是由一个或多个语素构成，语素主要分成两类：
- ▶ 词根 (Lemma) 也称为原形或字典形，是指能在字典中查到的语素，通常是一个词最主要的语素。承载核心语义且可独立成词。
- ▶ 词缀 (Affix) 是其他附着在原形上的语素，帮助在原形基础上衍生出新词，包含前缀、中缀、后缀等。

词的形态学

例如：

- ▶ 英语单词 unhappy 中，happy 为原形，-un 为前缀
- ▶ 邦托克语单词 fumikas（是强壮的）中，fikas（强壮）为原形，-um-为中缀
- ▶ 俄语单词 barabanshchik（鼓手）中，baraban（鼓）为原形，-shchik 为后缀

- ▶ 有些语言的单词通常只包含一个或者两个语素，但是有一些语言的单词则包含多达十个以上的语素。
- ▶ 汉语中每个单词的语素都很少，也不会根据性、数、格、人称等发生形态变化。特点：
 1. 语素数量少，通常 1-2 个。
 2. 无屈折变化（如名词无复数，动词无时态标记）。
 3. 依赖复合词（如“手机”=“手”+“机”）表达复杂概念。

英语中常见词形变换

词形变化	说明	举例
屈折 Inflection	通过“词根 + 词缀”的方式构成和原形“同一类型”的词	名词后加 -s 后缀 复数名词 (cat+s) 动词后加 -ed 后缀 动词的过去式 (walk+ed)
派生 Derivation	通过“词根 + 词缀”的方式构成和原形“不同类型”的词	employ 添加后缀 -ee 变为 employee meaning 添加后缀 -less 变为 meaningless
复合 Compounding	通过组合多个词根构成一个新词, 也称组合词	home + work → homework water + proof → waterproof
附着 Cliticization	通过“词根 + 附着语”的方式“附着”在词根上	I'm 中的'm 代表 am 附着在 I 上 We're 中're 代表 are
截搭 Blending	两个词语各自的一部分拼接起来构成新词	smoke (烟) + fog (雾) → smog (烟雾) spoon (勺子) + fork (叉子) → spork (叉勺)
缩略 Acronym	短语中多个单词首字母组合在一起组合成词	NLP 代表 Natural Language Processing IT 代表 Information Technology
截短 Clipping	将长的单词截为较短的单词	demonstration 简化为 demo refrigerator 简化为 fridge

通过语素组成词汇也可以反映语言的一个重要特性: 创造性。我们可以理解从未见过的词, 也可以通过新颖的方法将语素结合起来创造新词。如果能够自动将词汇分解为语素, 可以更好地对词汇进行进一步的分析。

词的词性

词性 (Part of Speech, POS) 也称词类, 是根据词在句子中扮演的语法角色以及与周围词的关系对词的分类。

例如:

- ▶ 表示事物的名字 (“钢琴”), 地点 (“上海”) 通常被归为名词。
- ▶ 而表示动作 (“踢”), 状态 (“存在”) 的词被归为动词。

词性分类目标: 语法结构分析: 构建句法树 (如依存句法分析)。

语义理解: 区分多义词 (如 “研究” 作名词或动词)。语言生成: 确保生成文本的语法正确性 (如形容词需修饰名词)。

词可以分为实义词 (Content Words) 和功能词 (Function Words):

- ▶ 实义词表达具体的意义。
由于实义词可以不断地增加, 因此这类词又被称作开类词 (Open class words)。实义词主要包含名词、动词、形容词等。
- ▶ 功能词则主要是为了满足语法功能需求。
由于功能词相对比较稳定, 一个语言中通常很少增加新的功能词, 因此功能词又被称作闭类词 (Close Class Words)。

词的词性

名词 (Noun) 是表示人、物、地点以及抽象概念的一类词。
例如：

- ▶ 1) 专有名词：Shanghai (上海) New York (纽约)
NLP 应用：命名实体识别 (NER) 的核心目标之一。
- ▶ 2) 类名词：city (城市) bird (鸟)
NLP 应用：文本分类、信息检索的关键特征。
- ▶ 3) 集体名词：family (家庭) army (军队)
NLP 挑战：单复数一致性 (如 “The team is/are working”)。
- ▶ 4) 物质名词：water (水) light (光)
NLP 挑战：不可数名词的量词搭配 (如 “一杯水”)。
- ▶ 5) 抽象名词：music (音乐) honesty (诚实)
NLP 应用：情感分析、主题建模的关键特征。

词的词性

动词 (Verb) 是表示动作或状态的一类词，是英语中最复杂的一类词。
例如：

- ▶ 1) Boys fly kites. (男孩们放风筝)
- ▶ 2) 不及物动词：Birds fly. (鸟会飞)
- ▶ 3) 连系动词：The rose smells sweet. (玫瑰花香)
- ▶ 4) 助动词：I may have meet him before. (我以前应该见过他)
- ▶ 5) 限定动词：John reads papers every day. (约翰每天都读论文)
- ▶ 6) 不限定动词：I hope to see you this morning. (我希望早上见到你)
- ▶ 7) 短语动词：Tom called up George. (汤姆给乔治打了电话)

句法分析与依存关系目标：识别动词及其与主语、宾语的关系。语义角色标注 (Semantic Role Labeling, SRL) 目标：标注动词的语义角色 (如施事、受事)。机器翻译中的动词对齐挑战：源语言与目标语言动词形态差异 (如英语 “run” 对应中文 “跑” 或 “经营”)。文本生成与摘要策略：优先保留核心动词 (如 “发布新产品”)。

词的词性

形容词 (Adjective) 是用来描写或修饰名词的一类词。

- ▶ 1) 简单形容词：
 - a) 由一个单词构成 good (好的) long (长的)
 - b) 由现在分词构成 interesting (令人感兴趣的)
 - c) 由过去分词构成 learned (博学的)
- ▶ 2) 复合形容词：duty-free (免税的) hand-made (手工制作的)
- ▶ 3) 限制性形容词：an Italian dish (一道意大利菜)
- ▶ 4) 描述性形容词：a delicious Italian dish (一道美味的意大利菜)

形容词在 NLP 中的关键应用

- ▶ 情感分析 (Sentiment Analysis) 目标：通过形容词判断文本情感极性 (如“好”表正向，“坏”表负向)。
- ▶ 文本分类与主题建模策略：形容词作为关键词特征 (如“科技”类文本高频词“先进”“高效”)。
- ▶ 机器翻译中的形容词对齐挑战：源语言与目标语言形容词形态差异 (如英文“interesting”对应中文“有趣的”)。方案：基于上下文对齐形容词单元。
- ▶ 文本生成与摘要策略：优先保留核心形容词 (如“重要的发现”)。技术：基于形容词重要性的句子压缩。

词的词性

副词 (Adverb) 是用来修饰动词、形容词、其他副词以及全句的词。

- ▶ 1) 简单副词: just (刚刚) only (仅仅)
- ▶ 2) 复合副词: somehow (不知怎地) somewhere (在某处)
- ▶ 3) 派生副词: interesting → interestingly (有趣地)
- ▶ 4) 方式副词: quickly (迅速) awkwardly (笨拙地)
- ▶ 5) 方向副词: outside (外面) inside (里面)
- ▶ 6) 时间副词: recently (最近) always (总是)
- ▶ 7) 强调副词: very (很) fairly (相当)

副词在 NLP 中的关键应用

- ▶ 情感分析 (Sentiment Analysis) 目标: 通过副词判断文本情感强度 (如 “非常好” 表强烈正向)。
- ▶ 文本分类与主题建模策略: 副词作为关键词特征 (如 “科技” 类文本高频词 “高效” “快速”)。
- ▶ 机器翻译中的副词对齐挑战: 源语言与目标语言副词形态差异 (如英文 “quickly” 对应中文 “快速地”)。
- ▶ 文本生成与摘要策略: 优先保留核心副词 (如 “迅速解决”)。

词的词性

数词 (Numeral) 是表示数目多少或者先后顺序的一类词。

表示数目多少的叫做基数词 (Cardinal numeral)。

表示顺序先后的叫做序数词 (Ordinal numeral)。

例如：

- ▶ 1) 基数词：one (1) nineteen (19)
- ▶ 2) 序数词：first (第一) fiftieth (第五十)

数词在 NLP 中的关键应用

- ▶ 命名实体识别 (NER) 目标：识别文本中的基数词与序数词 (如 “2023 年” “第三名”)。
- ▶ 信息抽取与问答系统策略：优先抽取数词 (如 “三个苹果”)。技术：基于数词的关键信息提取。
- ▶ 机器翻译中的数词对齐挑战：源语言与目标语言数词形态差异 (如英文 “twenty-one” 对应中文 “二十一”)。方案：基于规则或统计模型对齐数词单元。
- ▶ 文本生成与摘要策略：优先保留核心数词 (如 “三个关键点”)。技术：基于数词重要性的句子压缩。

词的词性

代词 (Pronoun) 是代替名词以及起名词作用的短语、子句和句子的一类词。代词的词义信息较弱，必须通过上下文来确定。例如：

- ▶ 1) 人称代词：
 - a) 主格：I, you, he, she, it, we, they
 - b) 宾格：me, you, him, her, it, us, them
- ▶ 2) 物主代词：
 - a) 形容词性物主代词：my, your, his, her, its, our, their
 - b) 名词性物主代词：mine, yours, his, hers, its, ours, theirs
- ▶ 3) 自身代词：myself, yourself, himself, herself, itself, ourselves, yourselves, themselves, oneself
- ▶ 4) 相互代词：each other, one another
- ▶ 5) 指示代词：this, that, these, those
- ▶ 6) 疑问代词：who, whom, whose, which, what
- ▶ 7) 关系代词：who, whom, whose, which, that, as
- ▶ 8) 不定代词：some, something, somebody, someone, any, anything, anybody, anyone, no, nothing, nobody, no one

词的词性

代词在 NLP 中的关键应用

- ▶ 指代消解 (Coreference Resolution) 目标：确定代词的具体指代对象 (如“他”指代“约翰”)。示例：句子“约翰跑了，他很快”中，“他”指代“约翰”。
- ▶ 信息抽取与问答系统策略：优先抽取代词指代的名词 (如“他选了第二”中的“他”)。技术：基于指代消解的关键信息提取。
- ▶ 机器翻译中的代词对齐挑战：源语言与目标语言代词形态差异 (如英文“he”对应中文“他”或“她”)。方案：基于上下文对齐代词单元。
- ▶ 文本生成与摘要策略：优先保留核心代词 (如“他跑了”)。技术：基于代词重要性的句子压缩。

词的词性

冠词 (Article) 是置于名词之前，说明名词所指的人或事物的一种功能词。

冠词不能够离开名词而独立存在。英语中冠词有三种冠词：

- ▶ 定冠词 (Definite article) “the”
- ▶ 不定冠词 (Indefinite article) “a/an”
- ▶ 零冠词 (Zero article)

冠词在 NLP 中的关键应用

- ▶ 机器翻译中的冠词对齐挑战：源语言与目标语言冠词形态差异（如英文 “the” 对应中文 “那本”）。方案：基于上下文对齐冠词单元。
- ▶ 文本生成与摘要策略：优先保留核心冠词（如 “the book”）。技术：基于冠词重要性的句子压缩。
- ▶ 信息抽取与问答系统策略：优先抽取冠词限定的名词（如 “the book”）。技术：基于冠词的关键信息提取。

词的词性

介词 (Preposition) 又称前置词，是用于表示名词或相当于名词的词语与句中其它词语的关系的一类词。介词在句子中不单独作为任何句子成分。介词后面的名词或者相当于名词的词语叫做介词宾语，与介词共同组合成介词短语。

- ▶ 1) 简单介词：at, in, of, since
- ▶ 2) 复合介词：as for, as to, out of
- ▶ 3) 二重介词：from under, from behind
- ▶ 4) 短语介词：according to, because of
- ▶ 5) 分词介词：including, regarding

介词在 NLP 中的关键应用

- ▶ 句法分析与依存关系目标：识别介词短语及其与动词、名词的关系（如“在桌子上”）。
- ▶ 信息抽取与问答系统策略：优先抽取介词短语（如“在早上”）。
技术：基于介词短语的关键信息提取。
- ▶ 机器翻译中的介词对齐挑战：源语言与目标语言介词形态差异（如英文“on”对应中文“在”）。方案：基于上下文对齐介词单元。
- ▶ 文本生成与摘要策略：优先保留核心介词短语（如“在早上”）。

词的词性

连词 (Conjunction) 是连接单词、短语、从句或句子的一类词。在句子中也不单独作为句子成分。

- ▶ 1) 简单连词：and, or, but, if
- ▶ 2) 关联连词：both ... and, not only ... but also
- ▶ 3) 分词连词：supposing, considering
- ▶ 4) 短语连词：as if, as long as, in order that
- ▶ 5) 并列连词：and, or, but, for
- ▶ 6) 从属连词：that, whether, when, because

连词在 NLP 中的关键应用

- ▶ 句法分析与依存关系目标：识别连词及其连接的成分（如“和”“或”）。
- ▶ 信息抽取与问答系统策略：优先抽取连词连接的成分（如“和”“或”）。技术：基于连词的关键信息提取。
- ▶ 机器翻译中的连词对齐挑战：源语言与目标语言连词形态差异（如英文“and”对应中文“和”）。方案：基于上下文对齐连词单元。
- ▶ 文本生成与摘要策略：优先保留核心连词（如“和”“或”）。

词的词性

感叹词 (Interjection) 是用来表示喜怒哀乐等情绪或情感的一类词。感叹词也没有实际意义，也不能在句子中构成任何句子成分，但是与全句有关联。

- ▶ Oh, it's you. 啊，是你
- ▶ Ah, how pitiful ! 呀，多可惜

感叹词在 NLP 中的关键应用

- ▶ 情感分析 (Sentiment Analysis) 目标：通过感叹词判断文本情感极性 (如“哇”表正向，“唉”表负向)。
- ▶ 文本分类与主题建模策略：感叹词作为关键词特征 (如“情感”类文本高频词“啊”“呀”)。
- ▶ 机器翻译中的感叹词对齐挑战：源语言与目标语言感叹词形态差异 (如英文“ Oh” 对应中文“哦”)。方案：基于上下文对齐感叹词单元。

词的词性

在语言学研究中，对于词性划分的标准、依据甚至目的等都存在大量分歧。到目前为止，还没有一个被广泛认可的统一划分标准。在不同的语料集中所采用的**划分粒度和标记符号**也都不尽相同。

- ▶ 英语宾州树库 (Penn TreeBank) 使用了 48 种不同的词性。
- ▶ 汉语宾州树库汉语词性被划分为 33 类。
- ▶ 布朗语料库 (Brown Corpus) 中则使用了具有 87 个词性。

词性划分的多样性与挑战

- ▶ 划分标准的分歧

语言学理论差异：不同语言学流派对词性的定义与分类标准不同（如结构主义、生成语法）。

语言特性差异：不同语言的词性系统差异显著（如英语的冠词、汉语的量词）。

- ▶ 划分目的的不同

理论研究：探索语言的内在规律（如句法结构、语义关系）。

应用需求：支持 NLP 任务（如句法分析、机器翻译）。

- ▶ 划分粒度的差异

粗粒度划分：如名词、动词、形容词等大类。

细粒度划分：如专有名词、普通名词、及物、不及物动词等。

2.2 词语规范化

词语规范化

- ▶ **词语规范化 (Word Normalization)** 任务是将单词或词形转化为标准形式，针对有多种形式的单词使用一种单一的形式进行表示。
- ▶ 在对自然语言文本进行分析前，通常需要对文本进行规范化的处理。文本的规范化处理主要包含句子切分、词语切分、词语规范化等步骤。
- ▶ 由于绝大部分语言的句子结束符数量有限，符号歧义性相对容易处理，因此句子切分通过词典结合模板或者有监督分类方法都可以达到较高的准确率。

词语切分

对于绝大部分的印欧语系语言来说，词语之间通常由分隔符区分开来。

- ▶ 英语是印欧语系（Indo-European languages）的典型代表，英语句子中绝大部分单词之间都由空格或标点分割。

例如：

输入：Let's first understand what's NLP.

输出：Let_'s_first_understand_what_'s_NLP_.

- ▶ 汉语为代表的汉藏语系（Sino-Tibetan languages）的语言中，单词之间通常没有分隔符。因此在对文本进行分析前，通常需要将句子切分为单词序列，称之为词语切分（Word Tokenization）。

词语切分

英语句子中绝大部分的单词可以通过空格和标点符号为分隔符进行识别，但是还是存在一些例外情况

- ▶ 例如：缩写 (Prof.)，区分句末句号与缩写句号 (如 U.S.A.)。日期 (02/18/2022)，格式多样性 (如 2022-02-18)。数字 (562,000) 保留数字完整性。连字符 (upper-case) 根据上下文判断整体或分开切分，注意语义完整性。
- ▶ 需要特别注意的是，上例中 Let's 被切分为 Let_'s

词形 (Token) 指的是在一个特定文档中的某个能够表达语义含义的字符序列，可能包含标点或符号。

大部分情况下词形和单词没有区别，但对于某些场景和算法有必要对单词和词形进行区分。

Deepseek 文档中的 token 是模型用来表示自然语言文本的基本单位，也是我们的计费单元，可以直观的理解为“字”或“词”；通常 1 个中文词语、1 个英文单词、1 个数字或 1 个符号计为 1 个 token。一般情况下模型中 token 和字数的换算比例大致如下：1 个英文字符 $\approx$ 0.3 个 token。1 个中文字符 $\approx$ 0.6 个 token。¹

¹https://api-docs.deepseek.com/zh-cn/quick_start/token_usage

词语切分

在英语中，一些特殊的符号和数字也需要完整的保留到一起。

- ▶ 结构化数据：比如数字 (67.20)、时间 (22:37)、微博话题标签 (# 卡塔尔 2022 年世界杯 #)、Email 地址 (cs_nlp@sufe.edu.cn) 等。
- ▶ 领域专有组合词：在特定的应用中有时也会将 Hong Kong、Head, Shoulders, Knees and Toes (歌词) 划分为一个词形

通常情况下针对英语等印欧语系语言的词语切分任务可以采用基于有限状态自动机 (Finite State Automaton, FSA) 融合正则表达式的方法完成。

有限状态自动机

有限状态自动机 (Finite State Automaton, FSA) 是一种用于建模系统行为的计算模型，广泛应用于自然语言处理 (NLP)

▶ 有限状态自动机的定义

有限状态：系统在任意时刻只能处于有限个状态中的一个。

状态转移：根据输入符号，系统从一个状态转移到另一个状态。

接受状态：某些状态被标记为“接受状态”，表示输入序列被成功处理。

▶ 形式化定义: 一个有限状态自动机可以表示为一个五元组：

$$M = (Q, \Sigma, \delta, q_0, F)$$

Q 有限状态集合。 Σ 输入符号的有限集合 (字母表)。 δ 状态转移函数： $\delta : Q \times \Sigma \rightarrow Q$ 。 q_0 初始状态： $q_0 \in Q$ 。 F 接受状态集合： $F \subseteq Q$ 。

▶ 有限状态自动机的类型

确定性有限状态自动机 (DFA)：每个状态对每个输入符号有且仅有一个转移。例如：输入：aab，状态转移： $q_0 \xrightarrow{a} q_1 \xrightarrow{a} q_2 \xrightarrow{b} q_3$ ，若 q_3 是接受状态，则输入被接受。

非确定性有限状态自动机 (NFA)：一个状态对同一个输入符号可能有多个转移。

有限状态自动机 I

```
class State:    #定义四个状态，描述分词器的状态机
    INITIAL = 0 #初始状态，等待识别新词素。
    NUMBER = 1  #识别数字
    EMAIL_LOCAL = 2 #识别电子邮件
    EMAIL_DOMAIN = 3 #识别电子邮件的域名

def tokenize_with_fsa(text):
    tokens = []
    current_token = []
    current_state = State.INITIAL
    i = 0 ## 字符索引，用于遍历输入文本

    while i < len(text):
        char = text[i] # 当前处理的字符
        # 根据当前状态执行不同逻辑
        if current_state == State.INITIAL:
            if char.isdigit() or char in '+-':
                current_state = State.NUMBER
                current_token.append(char)
                i += 1 # 进入新状态时立即递增索引
```

有限状态自动机 II

```
elif char.isalpha() or char == '_':
    current_state = State.EMAIL_LOCAL
    current_token.append(char)
    i += 1 # 进入新状态时立即递增索引
else:
    if current_token:
        tokens.append(''.join(current_token))
        current_token = []
    tokens.append(char)
    i += 1
    continue

if current_state == State.NUMBER:
    if char.isdigit() or char in '.eE':
        current_token.append(char)
        i += 1
    elif char == ':':
        # 允许时间格式 (如 22:37)
        current_token.append(char)
        i += 1
```

有限状态自动机 III

```
        else:
            tokens.append(''.join(current_token))
            current_token = []
            current_state = State.INITIAL
        continue

    if current_state == State.EMAIL_LOCAL:
        if char.isalnum() or char in '._%+-':
            current_token.append(char)
            i += 1
        elif char == '@':
            current_token.append(char)
            current_state = State.EMAIL_DOMAIN
            i += 1
        else:
            tokens.append(''.join(current_token))
            current_token = []
            current_state = State.INITIAL

    elif current_state == State.EMAIL_DOMAIN:
        if char.isalnum() or char in '.-':
```


有限状态自动机 IV

```
        current_token.append(char)
        i += 1
    else:
        tokens.append(''.join(current_token))
        current_token = []
        current_state = State.INITIAL

    if current_token:
        tokens.append(''.join(current_token))
    return tokens
```

测试用例

```
text = "Pay $67.20 to cs_nlp@sufe.edu.cn by 22:37"
```

```
print(tokenize_with_fsa(text))
```

```
# 输出: ['Pay', ' ', '$', '67.20', ' ', ' ', 'to', ' ', ' ', ' ',
        'cs_nlp@sufe.edu.cn', ' ', ' ', ' ',
        'by', ' ', ' ', '22:37']
```

正则表达式 I

```
import re
class Tokenizer:
    def __init__(self):
        self.patterns = [
            (r'[-+]?\\d+\\.?\\d*([eE][-+]?\\d+)?', 'NUMBER')
            , # 数字
            (r'\\d{1,2}:\\d{2}(?:\\s?[APap][Mm])?', 'TIME')
            , # 时间
            (r'#[\\w\\u4e00-\\u9fa5]+' , 'HASHTAG') ,
            # 中文标签
            (r'\\w+@\\w+\\.\\w+(?:\\.\\w+)*' , 'EMAIL') ,
            # 邮箱
            (r'\\b[A-Z][a-z]+\\b(?:[A-Z][a-z]+)*' , 'PROPER')
            # 专名连写
        ]

    def tokenize(self, text):
        tokens = []
        pos = 0
        while pos < len(text):
```

正则表达式 II

```
match = None
for pattern, tag in self.patterns:
    regex = re.compile(pattern)
    match = regex.match(text, pos)
    if match:
        tokens.append((match.group(0), tag))
        pos = match.end()
        break
if not match:
    if text[pos].isspace():
        pos += 1
    else:
        punct_match = re.match(r'[\w\']+ ',
                                text[pos:])
        if punct_match:
            tokens.append((punct_match.group(0), 'WORD'))
            pos += punct_match.end()
        else:
            tokens.append((text[pos], 'PUNCT
                            '))
```

正则表达式 III

```
                pos += 1
    return tokens
```

使用示例

```
tokenizer = Tokenizer()
text = "Order #2022世界杯 at 15:30 to cs_nlp@sufe.edu.cn"
print(tokenizer.tokenize(text))
# 输出: [('Order', 'WORD'), ('#2022世界杯', 'HASHTAG'),
        ('at', 'WORD'),
        ('15:30', 'TIME'), ('to', 'WORD'), ('
        cs_nlp@sufe.edu.cn', 'EMAIL
        ')]
```

词形还原

词形还原 (Lemmatization) 是将词的各种变化形式还原其词根的过程。通过词形还原可以实现词语的规范化，单词的不同变化形式统一为词根。

例如：

- ▶ 原始输入句：They are working on interesting tasks
- ▶ 词形还原后：they be work on interesting task

词形分析 (Morphological Parsing) 是将一个词分解成为语素的过程。最简单的方法是词典查表法，将每一个词的所有词形变换都存储下来，使用时直接匹配查找。

对于英语来说，构造包含所有绝大多数词形的词典能够有效地支撑许多应用场景。由于用词方式的变化和新词的不断出现，需要对该字典进行及时维护。

典型的词形分析算法包括基于**有限状态转换机 (Finite State Transducer)**方法，融合词典和有限状态转换机的方法以及统计机器学习方法等。

有限状态转换机

有限状态转换机 (Finite State Transducer, FST) 的功能：将输入符号序列转换为输出符号序列。

► 形式化定义:

$$T = (Q, \Sigma, \Gamma, \delta, \lambda, q_0, F)$$

Q 有限状态集合。 Σ 输入符号的有限集合 (字母表)。 Γ 输出符号的有限集合。 δ 状态转移函数: $\delta : Q \times \Sigma \rightarrow Q$ 。 λ 输出函数: $\lambda : Q \times \Sigma \rightarrow \Gamma^*$ 。 q_0 初始状态: $q_0 \in Q$ 。 F 接受状态集合: $F \subseteq Q$ 。

► 特点

输入：符号序列 (如字符串)。

输出：符号序列 (如翻译结果)。

应用：形态分析、机器翻译、语音识别。

有限状态转换机 I

```
class State:
    Q0 = 0 # 初始状态
    Q1 = 1 # 接受状态

def transition(state, char):
    if state == State.Q0:
        if char == 's':
            return State.Q1, '' # 输出空字符
        else:
            return State.Q0, char # 输出原字符
    return State.Q0, ''

def run_fst(input_string):
    current_state = State.Q0
    output = []
    for char in input_string:
        current_state, out_char = transition(
            current_state, char)
        output.append(out_char)
    return ''.join(output)
```

有限状态转换机 II

```
# 测试用例  
print(run_fst("cats")) # 输出: "cat"  
print(run_fst("dogs")) # 输出: "dog"
```


词干提取

词干提取 (Stemming) 是词形分析的简化版本，其目标是将具有词形变化（通常是屈折或派生）的词语还原为其词干（Word Stem）

例如：

- ▶ 词干提取算法 Porter Stemmer
- ▶ argue, argued, argues, arguing, 以及 argus 都转换为 argu.

与词形分析不同，词干提取并不要求还原的词干一定与其语言学词根完全一致，只需要将相关的单词映射为统一的词干。

词干提取

最简单的词干提取算法可以通过查询词表的方法获得，这种方法依赖词典所能覆盖的单词数量，并且需要及时更新以应对不断出现的新词。

另外一种方法是**后缀剥离 (Suffix-stripping)**，通过定义一组规则，将特定的后缀从词形中删除

例如：

- ▶ 如果单词以 ed 结尾, 则删除 ed
- ▶ 如果单词以 ing 结尾, 则删除 ing
- ▶ 如果单词以 ly 结尾, 则删除 ly

后缀剥离方法虽然可以很好地处理词语的规则变形，但是无法处理特殊变形（如：ran, took 等）。

词干提取

后缀替代 (Suffix Substitution) 算法将单词后缀替换为另外一个后缀。

例如：

- ▶ 如果单词以 ‘ational’ 结尾, 则替换为 ‘ate’ (relational → relate)
- ▶ 如果单词以 ‘ing’ 结尾, 则替换为 ‘ ’ (working → work)
- ▶ 如果单词以 ‘zzes’ 结尾, 则替换为 ‘z’ (quizzes → quiz)

Porter Stemmer 就采用了这种后缀替换的方法进行词干提取。

Porter Stemmer

Porter Stemmer 是一种用于英语词干提取 (Stemming) 的算法, 由 Martin Porter 于 1980 年提出。它的目标是将单词的不同形态 (如复数、时态、比较级等) 还原为统一的词干形式, 从而简化文本分析任务。

Porter Stemmer 通过 5 个阶段逐步裁剪词缀, 每个阶段针对特定类型的后缀:

- ▶ 处理复数、过去式等后缀。如 "caresses" → "caress" (去除 -es), "ponies" → "poni" (替换 -ies 为 -i)。
- ▶ 处理副词和形容词后缀。如 "relational" → "relate" (去除 -tional), "conditional" → "condition"。
- ▶ 处理以 -y 结尾的单词。如 "happy" → "happi" (保留辅音后的 -y)。
- ▶ 处理动词时态 (双辅音结尾)。如 "controlling" → "control" (裁剪 -ling)。
- ▶ 处理特殊后缀和优化。如 "synthesis" → "synthesi" (优化 -sis 结尾)。
- ▶ 每个阶段通过 ** 辅音-元音序列 (C-V 模式) ** 判断词干边界, 辅音序列数量 (m 值) 决定是否应用规则

Porter Stemmer I

```
# pip install nltk

from nltk.stem import PorterStemmer

# 初始化 PorterStemmer
stemmer = PorterStemmer()

# 示例单词
words = ["running", "cats", "happily", "jumps", "easily"]

# 应用词干提取
stemmed_words = [stemmer.stem(word) for word in words]
print(stemmed_words)

# 输出: ['run', 'cat', 'happili', 'jump', 'easili']
```

2.3 中文分词

中文分词

- ▶ 以英语为代表的印欧语系中词之间通常有分隔符（空格等）来区分，词可以比较容易地从句子中分割得到。
- ▶ 以汉语为代表的汉藏语系，以及以阿拉伯语为代表的闪-含语系（Semitic languages）中却不包含明显的词之间的分隔符，而是由一串连续的字符构成。
- ▶ 因此，针对汉语等语言的处理算法通常首先需要进行词语切分。

中文分词概述

中文分词 (Chinese Word Segmentation, CWS) 是指将连续字序列转换为对应的词序列的过程，也可以看做在输入的序列中添加空格或其他边界标记的过程。

例如：

- ▶ 上海财经大学是国际知名具有鲜明财经特色的高水平研究型大学
- ▶ 分词结果：上海财经大学 | 是 | 国际 | 知名 | 具有 | 鲜明 | 财经 | 特色 | 的 | 高水平 | 研究型 | 大学

由于汉语中语素绝大部分是单个汉字，很多情况下单独使用时也是词语，不单独使用时又是构词成分，这使得汉语构词具有很大的灵活性和很强的组词能力。

中文分词概述

正是因为汉语的这些特点，中文分词任务面临了巨大的挑战，主要困难来自以下三个方面：**分词规范、歧义切分和未登录词识别**。

汉语中对词的具体界定目前还没有定论。1992 年国家标准局颁布的《信息处理用现代汉语分词规范》中大部分规定都是通过举例和定性描述来体现。

例如：

- ▶ “二字或三字词，以及结合紧密、使用稳定的二字或三字词组，一律为分词单位。”
- ▶ 在实际应用中对“紧密”与“稳定”都很难界定，不可直接用于计算。

分词规范

北京大学计算语言研究所为了构造包含 2600 多万字《人民日报》基本标注语料库，制订了词语切分和词性标注规范《北京大学语料库加工规范》。针对国家标准分词规范，对分词单位进行了定义和解释。

- (1) 人名 (nr)：汉族方式的“姓”和“名”单独切分，，“姓”标注为 nrf，“名”标注为 nrg。例如：李/nrf 明/nrg，欧阳/nrf 洪涛/nrg；
- (2) 地名 (ns)：国名不论长短，作为一个切分单位，地名后有“省”、“市”等单字的现代行政区划名称时，不切分开，如果地名后的行政区划有两个以上的汉字，则将地名同行政区划名称切开。例如：中华人民共和国/ns，上海市/ns，[深圳/ns 特区/n]ns；
- (3) 机构名 (nt)：一般是短语型的，较长，且含有地名或人名等专名，按照参考文献 [47] 给出的规范需要先切分，再组合，加方括号标注为 nt。例如：[中国/ns 中文/n 信息/n 学会/n]nt，[复旦/ns 大学/n]nt；
- (4) 数词与数量词组：基数、序数、小数、分数、百分数一律不予切分，约数，前加副词、形容词或后加“来、多、左右”等助数词的应予切分。例如：一百二十三/m，约/d 一百/m 多/m 万/m；
- (5) 时间词：年月日时分秒，按年、月、日、时、分、秒切分，“牛年、虎年”等一律不予切分，标注为 t。例如：2021 年/t 9 月/t 16 日/t，牛年/t；
- (6) 成语习语：四个字的成语或习用语为一个切分单位，除标注其词类标记 i 或 l 外，还要求根据其在句子中的功能进一步标注子类，超过四个字的成语或习用语，一般不予切分，不分子类。例如：胸有成竹/iv，近水楼台先得月/i；

切分歧义

由于汉语构词方式的灵活性，使得同一个汉语句子很可能产生多个不同的分词结果，这些不同的分词结果也被称为切分歧义。

例如：

- ▶ 南京市长江大桥
- ▶ 切分方式 1：南京市 | 长江大桥
- ▶ 切分方式 2：南京 | 市长 | 江大桥

切分歧义

交集型切分歧义是指汉字串 AJB 中，AJ、JB 都可以分别组成词汇，则汉字串 AJB 被称为交集型切分歧义，此时汉字串 J 称作交集串。

例如：

- ▶ 乒乓球拍卖完了。
- ▶ 切分方式 1：乒乓 | 球 | 拍卖 | 完 | 了 |。
- ▶ 切分方式 2：乒乓 | 球拍 | 卖 | 完 | 了 |。

交集型切分歧义也被称为偶发歧义，当两个有交集的词“偶然”的相邻出现时这样的歧义才会发生。

切分歧义

组合型切分歧义是指如果汉字串 AB 满足 A, B, AB 同时为词, 则汉字串 AB 被称为组合型切分歧义。

例如：

- ▶ 他马上过来。
- ▶ 切分方式 1：他 | 马上 | 过来 |。
- ▶ 切分方式 2：他 | 马 | 上 | 过来 |。

组合性切分歧义也称为固有歧义，指的是词固有的属性，不依赖于“偶然”发生的上下文。

切分歧义

真歧义是指如果汉字串 ABC 满足多种切分方式下语法和语义均没有问题，只有通过上下文环境才能给出正确的切分结果，则汉字串 ABC 被称为真歧义。

例如：

- ▶ 白天鹅在水里游泳。
- ▶ 切分方式 1：白天 | 鹅 | 在 | 水 | 里 | 游泳 |。
- ▶ 切分方式 2：白天鹅 | 在 | 水 | 里 | 游泳 |。

对于这个句子来说，以上两种切分方式在语法和语义上都是正确的，需要考虑句子上下文环境，甚至是篇章内容才能进行正确判断。

未登录词

未登录词 (Out Of Vocabulary, OOV) 又称生词 (Unknown Words), 是指在训练语料中没有出现或者词典当中没有, 但是在测试数据中出现的词。

- ▶ 基于词典的分词方法所指的未登录词是所依赖的词典中没有的单词。
- ▶ 对于完全基于统计机器学习的方法, 未登录词是指训练语料中没有出现的单词。
- ▶ 对于融合词典特征的统计机器学习方法, 未登录词是指训练语料和词典中均未出现的词。

未登录词

汉语具有很强的灵活性，未登录词的类型也十分复杂，可以粗略地将汉语文本中常见的未登录词分为以下类型：

- ▶ 新出现的普通词汇：语言的使用会随着时代的变化而演化出新的词，这个过程在互联网环境中显得更为快速。
例如：场所码，二十大，精准防控，真香等。
- ▶ 命名实体 (Named Entity)：人名、地名、机构名等。
- ▶ 专业名词：出现在专业领域的词语 (例如：甲流，胞质溶胶)。
- ▶ 其他专有名词：新出现的产品名、电影名、书籍名等。

宗成庆教授在新闻领域的语料也进行了分词错误分析，结果发现未登录词造成的分词错误超过 98%，其中由命名实体引起的分词错误占到了 55% 左右。

基于词典的中文分词

基于词典的分词方法：把一个句子扫描一遍，遇到字典里有的词就标识出来，遇到复合词（比如“上海财经大学”）就找最长的词匹配，遇到不认识的字串就分割成单字词。

例如：

► “中国的发展”

从左到右扫描，先遇到“中”这个字，它本身是一个单字词，可以在这里做一个切割，当再遇到“国”字时，发现它可以和前面的“中”字组成一个更长的词，因此，就将分割点放在“中国”的后面。接下来，发现“中国”不会和后面的字组成更长的词，这个分割点就得以确定。

主要包含前向最大匹配，后向最大匹配以及双向最大匹配等三类。这些算法试图根据给定的词典，利用贪心搜索策略找到分词方案。

正向最大匹配方法

正向最大匹配方法 (Forward Maximum Matching, FMM)

- ▶ 设自动分词词典中的最长词条所含汉字个数为 m ，则取被处理材料当前字符串序数中的 m 个字作为匹配字段，查找分词词典。
- ▶ 若词典中有这样的一个 m 字词，则匹配成功，匹配字段作为一个词被切分出来；
- ▶ 如果词典中找不到这样的一个 m 字词，则匹配失败。
- ▶ 匹配字段去掉最后^{最后}一个汉字，剩下的字符作为新的匹配字段，进行新的匹配，如此进行下去，直至切分成功为止。即完成一轮匹配切分出一个词，然后再按上面的步骤进行下去，直到切分出所有的词为止。

正向最大匹配方法

例如：“计算机科学和工程”

- ▶ 假设词典中最长的词为 $m = 7$ 字；
- ▶ 先取“**计算机科学和工**”为匹配字段，查找分词词典以匹配这个字段。由于词典中没有该词，故匹配失败；
- ▶ 去掉最后一个汉字成为“**计算机科学和**”作为新的匹配字段，查找分词词典以匹配这个字段，同样匹配失败；
- ▶ 再取“**计算机科学**”作为新的匹配字段进行查找，假定词典中有“**计算机科学**”一词，则匹配成功，切分出第一个词“**计算机科学**”；
- ▶ 用同样的步骤还可以切分出第二，第三个词等。

正向最大匹配方法

```
def segword_fmm(test,dictionary,max_length):  
    my_list = []  
    len_hang = len(test)  
    while len_hang>0 :  
        tryWord = test[0:max_length]  
        while tryWord not in dictionary:  
            if len(tryWord)==1:  
                break  
            tryWord=tryWord[0:len(tryWord)-1]  
        my_list.append(tryWord)  
        test = test[len(tryWord):]  
        len_hang = len(test)  
    return my_list  
  
dics = ["南京","市长","长江","大桥"]  
test = "南京市长江大桥"  
dic = list(set(dics))  
segword_fmm(test,dic,5)  
#['南京', '市长', '江', '大桥']
```

正向最大匹配方法

存在的问题：较难处理交叉歧义和组合歧义

例如：

- ▶ “市场中国有企业才能发展”
- ▶ 切分结果为 “市场/中国/有/企业/才能/发展/”
- ▶ 可以看到，有两个分词错误：
 - (1) 交叉歧义 “中国/有” ；
 - (2) 组合歧义 “才能”。

FMM 方法思想简单，但是错误切分率较大错误切分率约为 1/169，一般不单独使用，而是与其他方法配合使用。

逆向最大匹配分词方法

逆向最大匹配分词方法 (Reverse Maximum Matching, RMM)

- ▶ 设自动分词词典中的最长词条所含汉字个数为 m ，则从被处理的文档末端开始匹配，每次取末端的 m 个字符作为匹配字段；
- ▶ 若词典中有这样的一个 m 字词，则匹配成功，匹配字段作为一个词被切分出来；
- ▶ 如果词典中找不到这样的一个 m 字词，则匹配失败。
- ▶ 匹配字段去掉最前面一个汉字，剩下的字符作为新的匹配字段，进行新的匹配，如此进行下去，直至切分成功为止。即完成一轮匹配切分出一个词，然后再按上面的步骤进行下去，直到切分出所有的词为止。

算法思想与 FMM 方法相似，不同的是：RMM 从句子的末尾开始处理，每次匹配不成功时去掉的是最前面的一个字。

逆向最大匹配分词方法

例如：“计算机科学和工程”

- ▶ 首先取“**计算机科学和工程**”作为匹配字段来查找分词词典，由于词典中没有该词，故匹配失败。
- ▶ 去掉最前面的一个汉字，即取“**算机科学和工程**”作为新的匹配字段进行查找，同样匹配失败，如此进行下去，
- ▶ 最后取“**工程**”作为匹配字段来查找分词词典，由于分词词典中有“**工程**”一词，则匹配成功，切分出第一个词“**工程**”。

逆向最大匹配分词方法

```
def segword_rmm(test,dictionary,max_length):  
    my_stack = []  
    len_hang = len(test)  
    while len_hang>0 :  
        tryWord = test[-max_length:]  
        while tryWord not in dictionary:  
            if len(tryWord)==1:  
                break  
            tryWord=tryWord[1:]  
        my_stack.append(tryWord)  
        test = test[0:len(test)-len(tryWord)]  
        len_hang = len(test)  
    return my_stack[::-1]  
dics = ["南京","市长","长江","大桥"]  
test = "南京市长江大桥"  
print(segword_rmm(test,dics,5))  
#['南京', '市', '长江', '大桥']
```


逆向最大匹配分词方法

例如：“市场中国有企业才能发展”

- ▶ 切分结果为：“市场/中/国有/企业/才能/发展”
- ▶ 有一个分词错误：组合歧义“才能”，应该切分为“才/能”。

RMM 方法的精度要高一些错误切分率约为 $1/245$ 。配合使用正向和逆向可以识别出分词中的交叉歧义，但对组合歧义却不能切分出来。

双向最大匹配法

双向最大匹配法 (Bi-direction Maximum Matching, Bi-MM)：
双向最大匹配法是将正向最大匹配法得到的分词结果和逆向最大匹配法的结果进行比较，从而决定正确的分词。

双向最大匹配的规则：

- ▶ (1) 如果正反向分词结果词数不同，则取分词数量少的那个；
- ▶ (2) 如果分词结果词数相同，则
 - ▶ 若分词结果相同，没有歧义，返回任意一个；
 - ▶ 若分词结果不同，返回其中单字数量较少的那个。

双向最大匹配法 I

```
import operator

def count_singleword(lista):
    count=0
    for elem in lista:
        if elem:
            count+=1
    return count

def segword_fmm(test,dictionary,max_length):
    my_list = []
    len_hang = len(test)
    while len_hang>0 :
        tryWord = test[0:max_length]
        while tryWord not in dictionary:
            if len(tryWord)==1:
                break
            tryWord=tryWord[0:len(tryWord)-1]
        my_list.append(tryWord)
        test = test[len(tryWord):]
        len_hang = len(test)
```

双向最大匹配法 II

```
    return my_list
def segword_rmm(test,dictionary,max_length):
    my_stack = []
    len_hang = len(test)
    while len_hang>0 :
        tryWord = test[-max_length:]
        while tryWord not in dictionary:
            if len(tryWord)==1:
                break
            tryWord=tryWord[1:]
        my_stack.append(tryWord)
        test = test[0:len(test)-len(tryWord)]
        len_hang = len(test)
    return my_stack[::-1]

def segword_bimm(test,dictionary,max_length):
    r_RMM = segword_rmm(test,dictionary,max_length)
    r_FMM = segword_fmm(test,dictionary,max_length)
    if len(r_RMM) > len(r_FMM):
        print(str(r_FMM))
    elif len(r_RMM) == len(r_FMM):
```

双向最大匹配法 III

```
if operator.eq(r_RMM,r_FMM):
    print(str(r_FMM))
else:
    l_RMM=count_singleword([len(e)==1 for e in
                             r_RMM])
    l_FMM=count_singleword([len(e)==1 for e in
                             r_FMM])

    if l_RMM > l_FMM:
        print(str(r_FMM))
    elif l_RMM == l_FMM:
        print(str(r_FMM) + ' 或 ' + str(r_RMM))
    else:
        print(str(r_RMM))
else:
    print(str(r_RMM))

dics = ["计算机","工程","科学"] #词典
test = "计算机科学与工程"
segword_bimm(test,dics,5)
```

基于词典的中文分词方法

基于词典的分词方法小结

- ▶ 双向最大匹配法在实用中得以广泛使用。
- ▶ 中文中 90%左右的句子，正向最大匹配法和逆向最大匹配法完全重合且正确。
- ▶ 只有大概 9%的句子两种切分方法得到的结果不一样，但其中必有一个是正确的（歧义检测成功）。
- ▶ 只有不到 1%的句子，或者正向最大匹配法和逆向最大匹配法的切分虽重合却是错的，或者正向最大匹配法和逆向最大匹配法切分不同但两个都不对（歧义检测失败）。
- ▶ 这类算法的优点是速度快，时间复杂度可以保持在 $O(n)$ ，且实现简单，效果尚可。
- ▶ 但对歧义和未登录词处理效果不佳。

基于线性链条件随机场的中文分词

将分词过程转换为对字的分类问题

- ▶ 对于输入句子中的每一个字 c_i ，根据它在分词结果中的位置赋予不同的标签：
- ▶ 开始 (B)、中间 (I)、结尾 (E) 以及单独成词 (S)

例如：

- ▶ 输入句子：他是研究生物化学的一位科学家。
- ▶ 分词结果：他 | 是 | 研究 | 生物化学 | 的 | 一 | 位 | 科学家 |。
- ▶ 对应标记：他/S 是/S 研/B 究/E 生/B 物/I 化/I 学/E 的/S 一/B 位/E 科/B 学/I 家/E 。/S

基于线性链条件随机场的中文分词

通过 BIES 标签可以将分词问题转换为字的分类问题。

- ▶ 一个字 c_i 的分类结果与其周边的字的分类结果有关联。
- ▶ 比如 c_i 被分类为 B 标签表示一个单词的开头时， c_{i+1} 标签就不应该分类为 S 标签表示一个成词的字。

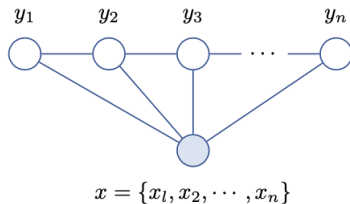
因此，中文分词任务也是典型的序列标注问题。可以采用条件随机场等机器学习方法进行解决。

基于线性链条件随机场的中文分词

条件随机场 (Conditional Random Field, CRF) 试图对多个变量在给定观测值后的条件概率进行建模。CRF 是一种判别式概率模型，用于对序列数据进行标注。

$x = \{x_1, x_2, \dots, x_n\}$ 为观测序列， $y = \{y_1, y_2, \dots, y_n\}$ 为对应的标记序列，条件随机场的**目标是构建条件概率模型 $P(y|x)$** 。判别式模型：直接建模 $P(y|x)$ ，而非联合概率 $P(x, y)$ 。序列建模：利用上下文信息（如前一个标记 y_{i-1} ）进行预测。特征函数：通过特征函数捕捉观测与标记之间的关系。

- ▶ 在中文分词任务中，观察序列 x 对应输入的字序列 $\{c_1, c_2, \dots, c_n\}$ ，标记序列为每个字对应的 BIES 标签。
- ▶ 在实际应用中，对序列任务进行建模时，通常使用线性链条件随机场 (Linear-chain CRF)。



基于线性链条件随机场的中文分词

条件随机场使用势函数和图结构上的团²来定义条件概率 $P(y|x)$ 。

给定观测序列 x ，线性链式条件随机场主要包含两种关于标记变量的团：单个标记变量 y_i 和相邻的标记变量 y_{i-1}, y_i 。选用指数势函数并引入特征函数 (Feature Function)，条件概率则定义为：

$$P(y | x) = \frac{1}{Z(x)} \exp \left(\sum_j \sum_{i=2}^n \lambda_j t_j (x, y_i, y_{i-1}, i) + \sum_k \sum_{i=1}^n \mu_k s_k (x, y_i, i) \right)$$
$$Z(x) = \sum_y \exp \left(\sum_j \sum_{i=2}^n \lambda_j t_j (x, y_i, y_{i-1}, i) + \sum_k \sum_{i=1}^n \mu_k s_k (x, y_i, i) \right)$$

$t_j(x, y_i, y_{i-1}, i)$ 是转移特征函数 (Transition feature function)，用于刻画相邻标记之间的相关关系以及观测序列对它们的影响。

$s_k(x, y_i, i)$ 是状态特征函数 (Status feature function)，用于刻画观测序列对标记变量的影响。

²在条件随机场中，团通过定义局部变量间的依赖关系，将复杂的全局联合概率分解为可计算的局部势函数乘积。极大团进一步简化了这一过程，使模型既能捕捉相邻变量的依赖，又能高效计算。

基于线性链条件随机场的中文分词

通常转移特征函数 t_j 和状态特征函数 s_k 的取值为 0 或 1，当满足特征条件时取值为 1，否则为 0。

线性链式条件随机场完全由特征函数 t_j, s_k 以及其对应的参数 λ_j 和 μ_k 决定。

针对中文分词任务，典型的转移特征如下：捕捉标记之间的依赖关系（如 B 后面不能接 S）。

$$t_j(x, y_i, y_{i-1}, i) = \begin{cases} 1 & \text{if } x_i = \text{财} \text{ 并且 } y_i = \text{B} \text{ 并且 } y_{i-1} = \text{E} \\ 0 & \text{otherwise} \end{cases}$$

表示第 i 个观测值为“财”时，相应的标记 y_i 和 y_{i-1} 很可能为 B 和 E。

基于线性链条件随机场的中文分词

典型的状态特征如下：捕捉字与标记之间的关系（如“中”字更可能是 B）。

$$s_j(x, y_i, i) = \begin{cases} 1 & \text{if } x_i = \text{上} \\ 0 & \text{otherwise} \end{cases}$$

表示第 i 个观测值为“上”时，相应的标记 y_i 很可能为 B。

基于线性链条件随机场的中文分词

如何设计有效的特征函数对于序列标注任务是至关重要的。

针对中文分词问题，可以使用模板的方式从当前字的上下文中构建。

模板名	描述
$c_k (k = -2, -1, 0, 1, 2)$	c_{i-2} 、 c_{i-1} 、 c_i 、 c_{i+1} 、 c_{i+2} 位置字符
$c_k c_{k+1} (k = -2, -1, 0, 1)$	$c_{i-2}c_{i-1}$ 、 $c_{i-1}c_i$ 、 $c_i c_{i+1}$ 、 $c_{i+1}c_{i+2}$ 双字组
$c_{-1}c_1$	$c_{i-1}c_{i+1}$ 双字组
$T(c_k) (k = -2, -1, 0, 1, 2)$	$T(c_{i-2})$ 、 $T(c_{i-1})$ 、 $T(c_i)$ 、 $T(c_{i+1})$ 、 $T(c_{i+2})$ 位置字符类型

图: 中文分词任务常用的模板。

其中 $T(c)$ 表示字符 c 的类型，包括阿拉伯数字、中文数字、标点符号、英文字母等。基于特征模板和训练语料，可以自动生成转移特征以及状态特征。

条件随机场的中文分词 I

```
import sklearn_crfsuite
from sklearn_crfsuite import metrics

# 特征提取函数
def word2features(sent, i):
    word = sent[i]
    features = {
        'bias': 1.0,
        'word': word,
        'word.isdigit()': word.isdigit(),
    }
    if i > 0:
        prev_word = sent[i-1]
        features.update({
            'prev_word': prev_word,
            'prev_word.isdigit()': prev_word.isdigit(),
        })
    else:
        features['BOS'] = True # Beginning of Sentence
    if i < len(sent)-1:
```

条件随机场的中文分词 II

```
        next_word = sent[i+1]
        features.update({
            'next_word': next_word,
            'next_word.isdigit()': next_word.isdigit(),
        })
    else:
        features['EOS'] = True    # End of Sentence
    return features

def sent2features(sent):
    return [word2features(sent, i) for i in range(len(
        sent))]

# 训练数据 (示例)
train_sents = [
    [('我', 'S'), ('们', 'S'), ('喜', 'B'), ('欢', 'I'),
     ('N', 'E'), ('L', 'S'), ('P', 'S')],
    [('自', 'B'), ('然', 'I'), ('语', 'B'), ('言', 'I'),
     ('处', 'B'), ('理', 'E')]
]
```

条件随机场的中文分词 III

提取特征

```
X_train = [sent2features([w for w, _ in sent]) for sent
              in train_sents]
y_train = [[label for _, label in sent] for sent in
              train_sents]
```

训练 CRF 模型

```
crf = sklearn_crfsuite.CRF(algorithm='lbfgs', c1=0.1, c2
                             =0.1, max_iterations=100)
crf.fit(X_train, y_train)
```

分词函数

```
def segment(text, crf_model):
    # 将输入文本转换为字列表
    chars = list(text)
    # 提取特征
    X_test = [sent2features(chars)]
    # 预测标签
    y_pred = crf_model.predict(X_test)[0]
    # 根据标签分词
```


条件随机场的中文分词 IV

```
words = []
current_word = ""
for char, label in zip(chars, y_pred):
    if label == 'B' or label == 'S':
        if current_word:
            words.append(current_word)
            current_word = char
        else:
            current_word += char
    if current_word:
        words.append(current_word)
return words
```

测试输入

```
text = "我们喜欢自然语言处理"
```

```
segmented_text = segment(text, crf)
```

```
print("分词结果:", segmented_text)
```

```
# 输出: ['我们', '喜欢', '自然', '语言', '处理']
```

基于感知器的中文分词

中文分词可以定义为将连续字序列转换为对应的词序列的过程。

- ▶ $x = \{c_1, c_2, \dots, c_n\}$ 表示输入字序列
- ▶ $y = \{w_1, w_2, \dots, w_m\}$ 表示输出词序列
- ▶ $F(x)$ 表示最优分词结果

中文分词可以形式化地表达为：

$$F(x) = \arg \max_{y \in \text{GEN}(x)} \text{SCORE}(y)$$

$\text{GEN}(x)$ 代表对于每一个输入句子 x 可能的所有候选输出。

$\text{SCORE}(y)$ 为针对分词结果 y 的评分函数。

将每一个分词后的单词序列 y 定义为一个特征向量 $\Phi(x, y)$ ，评分函数 $\text{SCORE}(y)$ 定义为由向量 $\Phi(x, y)$ 与参数 $\alpha \in \mathbb{R}^d$ 间的点积：

$$\text{SCORE}(y) = \Phi(x, y) \cdot \alpha$$

基于感知器的中文分词

将中文分词任务转化为上述问题后，需要解决如下三个问题：

- ▶ 问题 1：词序列预测在给定模型参数 α 和输入字序列 $x = \{c_1, c_2, \dots, c_n\}$ 的情况下，如何得到最优的词序列 $y = \{w_1, w_2, \dots, w_m\}$ ，即模型解码算法。
- ▶ 问题 2：模型参数学习在给定训练语料 x_i, y_i 的情况下，如何调整模型参数 α ，使其能针对训练语料得到最好的分词结果，即模型参数学习算法。
- ▶ 问题 3：特征定义给定输入字序列 x 和分词后的词序列 y ，如何定义特征对词序列进行描述，并能够区分分词序列的优劣，即构建特征向量 $\Phi(x, y) \in \mathbb{R}^d$ 。

基于感知器的中文分词

问题 1：模型解码算法，如何得到最优的词序列。

- ▶ 使用**集束搜索 (Beam Search)**算法进行解码：一种常用的限制搜索空间的启发式算法，在每一步解码过程中，从上一步解码的所有候选集中选取前 K 个得分最高的结果继续解码，而舍弃得分排在第 K 名之后的所有候选结果。
- ▶ 基本思路：针对输入的句子 x ，解码器每次读入一个字 c_i ，根据候选词队列，采用两种方法扩充候选结果：(1) 作为一个词的开始；(2) 添加到上一个候选词的末尾。对现有的候选分词结果进行评分，保留得分最高的前 K 个候选分词结果。重复上述过程，直到句子结束，输出得分最高的分词结果。
- ▶ 集束搜索可以理解作为一种“松弛”过的贪心算法，它并不能保证一定会得到得分最高的候选序列。

基于感知器的中文分词

问题 2：模型参数学习算法，调整模型参数 α ，得到最好的分词结果。

- ▶ 使用感知器算法进行训练：对训练语料中每一个句子，根据现有模型参数进行解码得到分词结果，与正确答案进行比对，如果结果错误则更新参数 α 。算法对整个训练语料迭代 T 轮。

```
for  $i = 1$  to  $T$  do //  $T$  轮迭代;  
    foreach  $(x, y) \in D$  do  
         $z = \arg \max_{y \in \text{GEN}(x)} \text{SCORE}(y)$ ;  
        if  $z \neq y$  then  
             $\alpha = \alpha + \Phi(x, y) - \Phi(x, z)$ ;  
        end  
    end  
end  
return  $\alpha$ 
```

- ▶ 也可采用平均感知器 (Average perceptron) 算法避免训练过程中的过拟合问题。

基于感知器的中文分词

问题 3：特征定义

- ▶ 针对特征向量 $\Phi(x,y)$ 的定义问题，感知器算法所需的特征由一系列人工选取的特征值组成，包含字、词以及长度信息。
- ▶ 在训练和解码时会使用特征模板将解码得到的序列映射到特征向量。

编号	模板	编号	模板
1	单词 w	8	所有单词的第一个与最后一个字符 c_1 和 c_2
2	二元单词 w_1w_2	9	字符 c 的前一个词 w
3	单字符单词 w	10	单词 w 之后的第一个字 c
4	初始字符 c 以及长度 l	11	两个连续单词的第一个字符 c_1 和 c_2
5	终止字符 c 以及长度 l	12	两个连续单词的最后一个字符 c_1 和 c_2
6	由空格隔开的字符 c_1 和 c_2	13	单词长度 l 以及之前的词 w
7	二元字符 c_1c_2	14	单词的长度 l 以及之后的单词 w

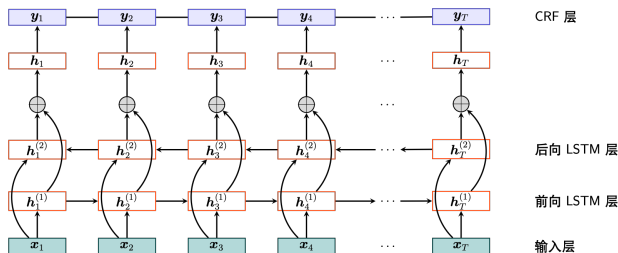
图: 基于感知器的中文分词算法所使用特征模板样例

- ▶ 基于感知器的方法可以使用词作为特征，而基于线性链条件随机场的方法只能使用字作为特征。

基于双向长短期记忆网络的中文分词方法

将分词过程转换为对字的分类问题

- ▶ 一个时刻的输出不但与过去某个时刻的信息相关，也与后续时刻的信息相关。
- ▶ BiLSTM 是由两层长短期记忆网络组成，它们结构相同但是信息传递的方向不同。
- ▶ BiLSTM 代替概率发射矩阵（观测概率矩阵），LSTM 可以捕获较长程的上下文信息，这样预测出来的发射概率会更准。
- ▶ CRF 层则可以学习到状态转移的约束条件。



中文分词评价方法

中文分词算法效果评测通常也采用统计机器学习算法评测中常用的指标进行对比，包括：精确率 (Precision, P)、召回率 (Recall, R)、F 值 (F-Measure)。

$$\text{精确率(P)} = \frac{\text{算法输出的正确分词结果个数}}{\text{算法输出的全部分词结果个数}} \times 100\%$$

$$\text{召回率(R)} = \frac{\text{算法输出的正确分词结果个数}}{\text{测试集合中全部答案个数}} \times 100\%$$

$$F_1 = \frac{2 \times P \times R}{P + R} \times 100\%$$

$$\text{未登录词召回率(R}_{\text{OOV}}) = \frac{\text{算法输出的未登录词正确结果个数}}{\text{测试集合中未登录词个数}} \times 100\%$$

中文分词语料库

中文分词算法的训练通常依赖大规模标注语料。

大规模中文分词语料集的建设也是推动中文分词算法快速发展的一个不可或缺的因素。SIGHAN 2005 和 SIGHAN 2008 是两组最常用的中文分词语料集合（SIGHAN 是国际计算语言学协会中文处理特别兴趣组）。

语料库名称	单词数量	简/繁体
北京大学分词语料库（PKU）	110 万	简体
香港城市大学分词语料库（CITYU）	145 万	繁体
微软研究院分词语料库（MSR）	237 万	简体
Academia Sinica（AS）	545 万	繁体
中文宾州树库 6.0（CTB 6.0）	78 万	简体
中文宾州树库 7.0（CTB 7.0）	120 万	简体
中文宾州树库 8.0（CTB 8.0）	162 万	简体
中文宾州树库 9.0（CTB 9.0）	208 万	简体
微博分词语料库（WordSeg-Weibo）	46 万	简体

图: 常用中文分词语料库

2.4 词性标注

词性标注

词性是词语的基本属性，根据其在句子中所扮演的语法角色以及与周围词的关系进行分类。

词性标注 (Part-of-speech Tagging, POS Tagging) 是指在给定的语境中确定句子中各词的词性。

词性标注的主要难点在于歧义性，即一个词可能在不同的上下文具有不同的词性。

- ▶ 兼类词多为常用词，而且越是常用词，其用法就越多。

例如：他们端正了态度（动词） | 他们的态度很端正（形容词）

努力学习社会科学（名词） | 这种方法比较科学（形容词）

- ▶ 北京大学计算语言学研究院 200 万字语料库统计，发现兼类词所占比例仅有 11%，但是出现的次数却达到了 47%。
- ▶ Brown 语料库的统计也发现超过 80% 的词通常只有一个词性。

还没有一个被广泛认可的统一词性划分标准，在不同的语料集中所采用的划分粒度和标记符号也都不尽相同。

词性标注

标签	描述	标签	描述
CC	并列连词	CD	数字
DT	限定词	EX	<u>there</u>
FW	外来词	IN	介词或从属连词
JJ	形容词	JJR	形容词比较级
JJS	形容词最高级	LS	列表项标记
MD	情态助动词	NN	名词单数
NNS	名词复数	NNP	专有名词单数
NNPS	专有名词复数	PDT	前限定词
POS	所有格结束词	PRP	人称代名词
PRP\$	物主代词	RB	副词
RBR	副词比较级	RBS	副词最高级
RP	小品词	SYM	符号
TO	to	UH	叹词
VB	动词	VBD	动词过去式
VBG	动词现在进行式	VBN	动词过去分词
VBP	动词一般现在式 非第三人称单数	VBZ	动词一般现在式 第三人称单数
WDT	Wh-限定词	WP	Wh-代词
WP\$	所有格 Wh-代词	WRB	Wh-副词

图: 宾州树库中的词性标签

基于规则的词性标注

利用词典和搭配规则针对词语和上下文进行分析，从而得到句子中每个词语词性。

Brill Tagger 是一种利用错误驱动方法学习转换规则的词性标注算法。其在 Brown 语料库上仅使用 71 个规则就得到接近 95% 的分析准确率。

- ▶ 初始化：对于词典中包含的词语，根据词语最常使用的词性设置初始值；对于词典中没有的单词根据词性分析结果设置初始值（例如：以大写字母开头的设置为专有名词）。
- ▶ 规则转换：根据补丁规则对初始标注进行转换，补丁规则包含：
 - (a) 如果某单词词性为 a，并且其所在上下文为 C，那么将其词性转换为 b；
 - (b) 如果某单词词性为 a，并且其具有词汇属性 P，那么将其词性转换为 b；
 - (c) 如果某单词词性为 a，并且其周边范围 R 内有一个词汇具有属性 P，那么将其词性转换为 b；

基于规则的词性标注

Brill Tagger根据补丁规则对初始标注进行转换

例如：

- ▶ 补丁规则“NN VB PREV-TAG TO”表示，如果一个单词被标注为了 NN（名词），并且它前面的单词标注为了 TO（不定式“to”），那么将这个单词的词性转换为 VB（动词）。
- ▶ 解决类似“to book a hotel”中对于单词 book 的词性默认标注错误的问题。

基于规则的词性标注

Brill Tagger 采用基于错误驱动的有监督模板学习方法学习补丁规则。

- ▶ 首先根据现有的初始词典和补丁模板对训练语料进行分析，将错误的分析结果汇总为三元组 $\langle \text{taga}, \text{tagb}, n \rangle$ 形式，表示一个单词的词性应该为 tagb，但是在评测语料中有 n 次都被标注为了词性 taga。
- ▶ 根据所得到的三元组，生成补丁规则需利用如下模板：
 - 前一个（或者后一个）单词被标注为了 z
 - 前面第二个（或者后面第二个）单词被标注为了 z
 - 前面两个（或者后面两个）单词某一个被标注为了 z
 - 前面三个（或者后面三个）单词某一个被标注为了 z
 - 前一个单词被标注为了 z，并且后一个单词被标注为了 w
 - 前一个单词被标注为了 z，并且前面第二个（或者后面第二个）单词被标注为了 w
 - 当前单词是（不是）首字母大写
 - 前一个单词是（不是）首字母大写

基于错误驱动的规则方法可以在一定程度上缓解人工规则抽取的时间成本和人力成本。在词性标注问题中取得了不错的效果。但是其效果严重依赖于训练语料的规模和质量，同时也较难处理未登录词。此外，受到规则模板复杂度的限制，其效果通常也低于基于统计机器学习的方法。

BrillTagger I

```
# import nltk
# nltk.download('brown') #下载数据
from nltk.tag import BrillTaggerTrainer, UnigramTagger
from nltk.corpus import brown
# 初始标注器
baseline_tagger = UnigramTagger(brown.tagged_sents(
                                categories='news'))
# 定义规则模板 (如检查前后词标签)
templates = nltk.tag.brill.fntbl37()
# 训练Brill Tagger
trainer = BrillTaggerTrainer(baseline_tagger, templates)
brill_tagger = trainer.train(brown.tagged_sents(
                                categories='news'),
                                max_rules=100)

# 测试标注
print(brill_tagger.tag("They are running a test".split())
      ))
```


基于隐马尔可夫模型的词性标注

隐马尔可夫模型 (Hidden Markov Model, HMM) 马尔可夫过程扩充而来的一种随机过程。HMM 是一种基于双重随机过程的序列建模方法，通过隐藏状态序列与观测序列的联合概率分布对时序数据建模。在序列标注任务中得到了广泛应用。

一个隐马尔可夫模型可用如下 5 个参数定义：

- ▶ **N：状态数**。所有的状态记为 $S = \{s_1, s_2, \dots, s_N\}$ ，系统在 t 时刻的状态记为 q_t ， $Q = \{q_1, q_2, \dots, q_T\}$ 为长度为 T 的状态序列。
- ▶ **M：观察值数**。所有的可能观察值记为 $V = \{v_1, v_2, \dots, v_M\}$ 。系统在 t 时刻的观测值记为 o_t 。 $O = \{o_1, o_2, \dots, o_T\}$ 为长度 T 的观测序列。
- ▶ **π ：初始状态概率**。 $\pi = [\pi_i]_{1 \times N}$, $\pi_i = P(q_1 = s_i)$, $1 \leq i \leq N$ ，表示初始时刻 $t = 1$ 时处于某个状态 s_i 的概率。
- ▶ **A：状态转移概率矩阵**。
 $A = [a_{ij}]_{N \times N}$, $a_{ij} = P(q_{t+1} = s_j | q_t = s_i)$, $1 \leq i, j \leq N$ ，表示在时刻 t 处于状态 s_i 的条件下，下一时刻 $t + 1$ 转移到状态 s_j 的概率。
- ▶ **B：观测概率矩阵**。
 $B = [b_j(k)]_{N \times M}$, $b_j(k) = P(o_t = v_k | q_t = s_j)$, $1 \leq j \leq N, 1 \leq k \leq M$ ，表示在时刻 t 处于状态 s_j 的条件下，观测到 v_k 的概率。

基于隐马尔可夫模型的词性标注

以天气模型为例，完整展示 HMM 参数的应用：

- ▶ 状态集合：

$S = \text{Rainy}, \text{Sunny}$, $N = 2$ 。

- ▶ 观测集合：

$V = \text{walk}, \text{shop}, \text{clean}$, $M = 3$ 。

- ▶ 初始概率：

$\pi = \text{Rainy}:0.6, \text{Sunny}:0.4$ 。

- ▶ 转移矩阵：

$A = \{$
 $\text{Rainy} \rightarrow \text{Rainy}:0.7, \text{Rainy} \rightarrow \text{Sunny}:0.3$
 $\text{Sunny} \rightarrow \text{Rainy}:0.4, \text{Sunny} \rightarrow \text{Sunny}:0.6 \}$

- ▶ 观测矩阵：

$B =$
 $\text{Rainy}: \{ \text{walk}:0.1, \text{shop}:0.4, \text{clean}:0.5 \}$
 $\text{Sunny}: \{ \text{walk}:0.6, \text{shop}:0.3, \text{clean}:0.1 \}$

基于隐马尔可夫模型的词性标注

为了简化起见，隐马尔可夫模型可以表示成 $\lambda = (A, B, \pi)$ 。M, N 也隐含地已经包含在 A, B, π 中。

隐马尔可夫模型的三个主要问题：

- ▶ 问题 1：观测概率计算。在给定模型 $\lambda = (A, B, \pi)$ 的情况下，如何根据观测序列 $O = \{o_1, o_2, \dots, o_T\}$ 计算 $P(O|\lambda)$ ，即在给定模型情况下，如何观测序列的概率。
- ▶ 问题 2：状态序列预测。在给定模型 $\lambda = (A, B, \pi)$ 和观测序列 $O = \{o_1, o_2, \dots, o_T\}$ 的情况下，如何得到与该观测序列最匹配的状态序列 $Q = \{q_1, q_2, \dots, q_T\}$ ，即如何根据观测序列推断出隐藏的状态序列。
- ▶ 问题 3：模型参数学习。在给定观测序列 $O = \{o_1, o_2, \dots, o_T\}$ 情况下，如何调整模型参数 $\lambda = (A, B, \pi)$ 使得该序列的 $P(O|\lambda)$ 最大，即如何训练模型使其能最好地建模观测序列。

基于隐马尔可夫模型的词性标注

针对词性标注任务，使用 HMM 可以按照如下方式构建和学习模型：

- ▶ N 为词性数， $S = \{s_1, s_2, \dots, s_N\}$ 为词性表，包含所使用到的所有词性信息。 M 为单词数， $V = \{v_1, v_2, \dots, v_M\}$ 为单词词表，包含所有单词。
- ▶ 给定一个由 T 个单词组成的句子 $W = w_1, w_2, \dots, w_T$ ，即相当于观测序列 $O = \{o_1, o_2, \dots, o_T\}$ ， o_i 为句子中第 i 个单词 w_i 。
- ▶ 状态序列 $Q = \{q_1, q_2, \dots, q_T\}$ 则表示输入句子中单词对应的词性。
- ▶ 根据训练语料，可以使用最大似然估计的 Baum-Welch 方法高效地得到模型参数。
- ▶ 在此基础上，针对输入的句子可以利用维特比（Viterbi）算法应用动态规划求解状态路径，从而得到对应的词性。

基于隐马尔可夫模型的词性标注

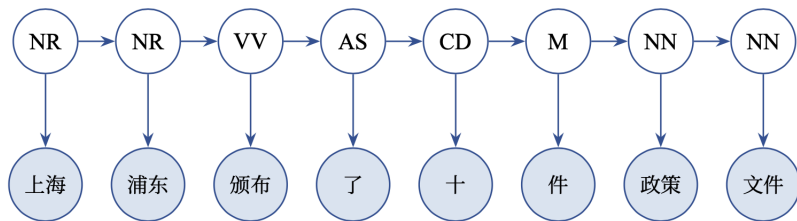


图: 基于词性标注的隐马尔可夫模型概率图模型样例

在实际应用过程中，使用隐马尔可夫模型进行词性标注，通常还需要解决两个问题：**长句子**和**未登录词**。

常会限定一个句子中单词的最大数量。

引入语法规则，对人名、地名、数词、副词等进行判断，还可以根据更大规模的未登录词的词性统计，从而设定更合理的观测概率。

HiddenMarkovModelTagger 处理英语 I

```
# HiddenMarkovModelTagger处理英语文本
# 下载brown语料库 https://www.nltk.org/nltk\_data/ 解压
# 到/Users/xxx/nltk_data/
# corpora

import nltk
from nltk.tag.hmm import HiddenMarkovModelTagger

# 自定义发射概率平滑函数
def smoothed_emission(word, tag, hmm_model,
                      external_probs, lambda_=0.7)
    :
    default_prob = hmm_model._emission_prob(tag, word)
    external_prob = external_probs.get((word, tag), 0.0)
    return lambda_ * default_prob + (1 - lambda_) *
        external_prob

# 加载预训练HMM模型
train_data = nltk.corpus.brown.tagged_sents(categories='
news')
```

HiddenMarkovModelTagger 处理英语 II

```
hmm_tagger = HiddenMarkovModelTagger.train(train_data)
# 定义外部统计 (示例)
external_probs = {
    ('AI', 'NN'): 0.8,
    ('AI', 'JJ'): 0.2,
    # 其他未登录词统计...
}
# 标注句子
sentence = "ChatGPT-4 is an AI developed by OpenAI in
            2023".split()
tagged = hmm_tagger.tag(sentence)
# 输出结果
print(tagged)
# [('ChatGPT-4', 'PPS'), ('is', 'BEZ'), ('an', 'AT'), ('
    AI', 'QL'), ('developed', '
    VBN'), ('by', 'IN'), ('
    OpenAI', 'CD'), ('in', 'IN')
    , ('2023', 'AT')]
```

HiddenMarkovModel 中文分词 I

```
import os
import pickle

class HMM(object):
    def __init__(self):
        # 用于存取算法中间结果，不用每次训练模型
        self.model_file = 'hmm_model.pkl' # 模型存储文件

        # 状态集合
        self.state_list = ['B', 'M', 'E', 'S'] # 状态集合：B-词首，M-词中，E-词尾，S-单字词

        self.load_para = False # 是否加载已有模型
        pass

    # 用于加载已计算的中间结果，当需要重新训练时，需要初始化清空结果
    def try_load_model(self):
```


HiddenMarkovModel 中文分词 II

```
self.A_dic = {} # 状态转移概率矩阵 (State
                  transition) (状态->状态的条
                  件概率)
self.B_dic = {} # 发射概率矩阵 (Emission) (状
                  态->词语的条件概率)
self.Pi_dic = {} # 初始状态概率
self.load_para = False

# 计算转移概率、发射概率以及初始概率
def train(self, path):
    self.try_load_model() # # 清空旧参数, 重置概率矩
                          阵
    Count_dic = {} # 统计状态出现的次数, 求p
                   (0)
    def init_parameters(): # 初始化参数
        for state in self.state_list:
            self.A_dic[state] = {s: 0.0 for s in
                                  self.state_list} #{'B': 0.0,
                                  'M': 0.0, 'E': 0.0, 'S': 0.
                                  0}
            self.Pi_dic[state] = 0.0
```

HiddenMarkovModel 中文分词 III

```
        self.B_dic[state] = {}
        Count_dic[state] = 0 # 初始化状态计数器

def make_label(text): #根据词语长度生成状态标签
                        序列
    out_text = []
    if len(text) == 1:
        out_text.append('S')
    else:
        out_text += ['B'] + ['M'] * (len(text) -
                                     2) + ['E'] ##['B', 'M', '
                                                M', ..., 'E']
    return out_text

init_parameters()
line_num = -1 # 行数计数器

# 观察者集合，主要是字及标点等
```

HiddenMarkovModel 中文分词 IV

```
words = set() #set() 函数创建一个无序不重复元素
                集，可进行关系测试，删除重复
                数据，还可以计算交集、差集、
                并集等。

with open(path, encoding='utf-8') as f:
    for line in f:
        line_num += 1

        line = line.strip() #strip()移除字符串头
                            尾指定的字符

        if not line:
            continue      #continue 语句跳出本次循
                            环，而break跳出整个循环。

        word_list = [i for i in line if i != ' ' ] #分割为单个汉字和标点符号
        words |= set(word_list) # 更新字的集合
                                ##set(word_list)去除集合中重
                                复的元素，然后|=求并集

        linelist = line.split()
```

HiddenMarkovModel 中文分词 V

```
line_state = []
for w in linelist:
    line_state.extend(make_label(w)) #
    生成每个词的状态序列
#
    print(len(word_list))
#
    print(len(line_state))
assert len(word_list) == len(line_state)
    ## 确保字符数与状态数一致

for k, v in enumerate(line_state):
    Count_dic[v] += 1 # 状态出现次数统计
    if k == 0:
        self.Pi_dic[v] += 1 # 初始状态
        计数
    else:
        self.A_dic[line_state[k - 1]][v]
        += 1 # 计算转移概率 # 状
        态转移计数
```

HiddenMarkovModel 中文分词 VI

```
        self.B_dic[line_state[k]][
            word_list[k]] = self.B_dic[
                line_state[k]].get(word_list
                    [k], 0) + 1 # 计算发射概率
                                # 发射计数

# 参数归一化
self.Pi_dic = {k: v * 1.0 / line_num for k, v in
                self.Pi_dic.items()} # 初始概率

self.A_dic = {k: {k1: v1 / Count_dic[k] for k1,
                  v1 in v.items()} for k, v in
               self.A_dic.items()} # 转移概率

# 加1平滑
self.B_dic = {k: {k1: (v1 + 1) / Count_dic[k]
                  for k1, v1 in v.items()} for
               k, v in self.B_dic.items()}

# 保存模型
with open(self.model_file, 'wb') as f:
    pickle.dump(self.A_dic, f)
```

HiddenMarkovModel 中文分词 VII

```
        pickle.dump(self.B_dic,f)
        pickle.dump(self.Pi_dic,f)
    return self

def viterbi(self, text, states, start_p, trans_p,
            emit_p):
    v = [{}] # 概率矩阵
    path = {} # 路径记录

    # 处理第一个字符的未登录词
    first_char = text[0]
    # 判断是否为未登录词：在四个状态的发射概率中均不存在
    never_seen = first_char not in emit_p['S'] and
                first_char not in emit_p['M'] \
                and first_char not in emit_p['B']
                and first_char not in emit_p
                ['E'] # 判断是否未登录词

    for y in states:
        # 未登录词：发射概率设为均匀分布（如1.0）
```

HiddenMarkovModel 中文分词 VIII

```
emit = emit_p[y].get(first_char, 1.0 if
                        never_seen else 0) # 未登录
                                           词处理
v[0][y] = start_p[y] * emit
path[y] = [y]

# 迭代处理后续字符
for t in range(1, len(text)):
    current_char = text[t]
    never_seen = current_char not in emit_p['S']
                  and current_char not in
                  emit_p['M'] \
                  and current_char not in emit_p['
                  B'] and current_char not in
                  emit_p['E']

    v.append({})
    newpath = {}

    for y in states:
        # 未登录词：发射概率设为均匀分布（如1.0
        )
```

HiddenMarkovModel 中文分词 IX

```
emit = emit_p[y].get(current_char, 1.0
                        if never_seen else 0)

# 计算所有可能前驱状态的最大概率路径
(prob, state) = max(
    [(v[t-1][y0] * trans_p[y0].get(y, 0)
      * emit, y0)
     for y0 in states if v[t-1][y0] > 0]
    or [(0.0, states[0])] # 保底逻辑
)
v[t][y] = prob
newpath[y] = path[state] + [y]

path = newpath # 更新路径

# 选择最终状态 (修正逻辑)
last_probs = [(v[-1][y], y) for y in ['E', 'S']]
# 句子应结束于E或S状态
(prob, state) = max(last_probs) if last_probs
else (0.0, 'S')
```


HiddenMarkovModel 中文分词 X

```
    return (prob, path[state])

def cut(self, text):
    if not text: # 空输入处理
        return []
    prob, pos_list = self.viterbi(text, self.
                                   state_list, self.Pi_dic,
                                   self.A_dic, self.B_dic) # 获取状态序列

    print(str(pos_list))
    begin, next = 0, 0
    for i, char in enumerate(text):
        pos = pos_list[i]
        if pos == 'B':
            begin = i # 记录词首位置
        elif pos == 'E':
            yield text[begin:i + 1] # 输出词语
            next = i + 1
        elif pos == 'S':
            yield char # 输出单字
```

HiddenMarkovModel 中文分词 XI

```
        next = i + 1
    if next < len(text):
        yield text[next:] # 处理剩余字符
```

#隐马尔可夫模型

```
hmm = HMM()
```

```
hmm.train('trainCorpus.txt_utf8')
```

```
text = '针对长句子和未登录词问题，HMM词性标注需结合工程
```

优化与语言资源（词法规则、外部统计）。通过规则与统计的混合策略，可显著提升标注鲁棒性，尤其在专业领域和低资源语言场景中表现突出。未来趋势将更多融合深度学习（如BERT上下文嵌入）与符号规则，实现更高精度的序列标注。'

```
res = hmm.cut(text)
```

```
print(str(list(res)))
```

基于卷积神经网络的词性标注

- ▶ 在深度神经网络应用于自然语言处理任务之前，绝大多数自然语言处理算法依赖于特征工程。
- ▶ Collobert 等人在 2011 年所提出的“从零开始的 NLP”³ 框架利用统一的具有多个隐藏层的神经网络解决了多个自然语言处理任务，省去了特征工程的步骤，推动了深度学习在自然语言处理任务中的快速发展。
- ▶ 参数为 θ 的神经网络 $f_{\theta}(\cdot)$ ，任意一个具有 L 层的前馈神经网络，可以看成是一个复合函数 $f_{\theta}^l(\cdot)$ ：

$$f_{\theta}(\cdot) = f_{\theta}^L(f_{\theta}^{L-1}(\dots f_{\theta}^1(\cdot)\dots))$$

³Ronan Collobert, et al. Natural Language Processing (Almost) from Scratch, Journal of Machine Learning Research, 2011

基于卷积神经网络的词性标注

将单词转换为特征向量

首先通过[表查询](#)： $LT_{\mathbf{W}}(w) = \langle \mathbf{W} \rangle_w^1$ 将单词通过表查询转换为词性向量表示，词向量的维度是 d_{wrd} 。

其中 $\mathbf{W} \in \mathbb{R}^{d_{\text{wrd}} \times |D|}$ ， D 是包含有限个单词的字典，

$\langle \mathbf{W} \rangle_w^1 \in \mathbb{R}^{d_{\text{wrd}}}$ 表示 $\mathbf{W}$ 矩阵的第 w 列。 $\mathbf{W}$ 矩阵也是要学习的参数。对于给定的任意一句包含 T 个单词的句子 $[w]_1^T$ ，通过表查询层对序列中的每个单词进行转换，得到如下表查询层输出矩阵：

$$LT_{\mathbf{W}} \left([w]_1^T \right) = \left(\langle \mathbf{W} \rangle_{[w]_1}^1, \langle \mathbf{W} \rangle_{[w]_2}^1 \dots \langle \mathbf{W} \rangle_{[w]_T}^1 \right)$$

基于卷积神经网络的词性标注

扩展到多个离散特征

除了单词本身之外，还可以提供一些其他特征，例如该单词在词典中最常见的词性等信息。因此，可以将单词更一般地表示为 K 个离散特征 $w = D_1 \times D_2 \times \dots \times D_K$ ， D_K 是第 k 维特征的字典。

$LT_{\mathbf{w}^k}(\cdot)$ 是每维特征的查询表， $\mathbf{w}^k \in \mathbb{R}^{d_{\text{wrd}}^k \times |D^k|}$ 是第 k 维特征的嵌入向量矩阵， $d_{\text{wrd}}^k \in \mathbb{N}$ 是用户给定的向量维度。对于一个单词 w ，其特征向量的维度 $d_{\text{wrd}} = \sum_k d_{\text{wrd}}^k$ ，通过表查询得到连接后的向量：

$$LT_{\mathbf{w}^1, \dots, \mathbf{w}^K}(w) = \begin{pmatrix} LT_{\mathbf{w}^1}(w_1) \\ \vdots \\ LT_{\mathbf{w}^K}(w_K) \end{pmatrix} = \begin{pmatrix} \langle \mathbf{w}^1 \rangle_{w_1}^1 \\ \vdots \\ \langle \mathbf{w}^K \rangle_{w_K}^1 \end{pmatrix}$$

基于卷积神经网络的词性标注

由此，可以得到如下表查询层输出矩阵：

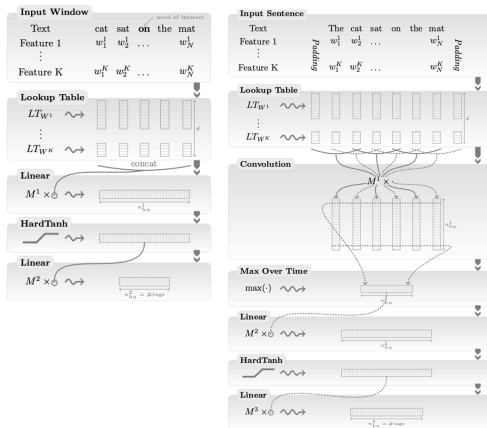
$$\mathbf{LT}_{\mathbf{w}^1, \dots, \mathbf{w}^K} \left([\mathbf{w}]_1^T \right) = \begin{pmatrix} \langle \mathbf{w}^1 \rangle_{[\mathbf{w}_1]_1}^1 & \cdots & \langle \mathbf{w}^1 \rangle_{[\mathbf{w}_1]_T}^1 \\ \vdots & & \vdots \\ \langle \mathbf{w}^K \rangle_{[\mathbf{w}_K]_1}^1 & \cdots & \langle \mathbf{w}^K \rangle_{[\mathbf{w}_K]_T}^1 \end{pmatrix}$$

基于卷积神经网络的词性标注

从单词特征向量中提取更高层特征

查找表层产生的特征向量需要在神经网络的后续层中组合，以便为句子中的每个单词生成标签决策。

考虑两种标记单词的常见方法：窗口方法和卷积方法。



基于卷积神经网络的词性标注

在表查询层后连接的是卷积层 (Convolutional Layer)，根据所设置的窗口大小 d_{win} ，将每个单词周边的单词拼接起来构成具有 $d_{wrd}d_{win}$ 维度的向量：

$$f_{\theta}^1 = \left\langle \text{LT}_W \left([W]_1^T \right) \right\rangle_t^{d_{win}} = \begin{pmatrix} \langle \mathbf{W} \rangle_{[w]_t - d_{win}/2}^1 \\ \vdots \\ \langle \mathbf{W} \rangle_{[w]_t}^1 \\ \vdots \\ \langle \mathbf{W} \rangle_{[w]_t + d_{win}/2}^1 \end{pmatrix}$$

边框影响：上式特征窗口没有很好地定义句子开头或结尾附近的单词。为了规避这个问题，用一个特殊的“**PADDING**”词来补充句子，在开头和结尾复制 $d_{win}/2$ 次。这类似于在序列模型中使用“start”和“stop”符号。

基于卷积神经网络的词性标注

f_{θ}^l 会被送入单层或者多层的卷积层，第 l 层的第 t 列向量可以根据如下公式计算得到:

$$\left\langle f_{\theta}^l \right\rangle_t^1 = \mathbf{W}^l \left\langle f_{\theta}^{l-1} \right\rangle_t^{d_{\text{win}}} + b^l \quad \forall t$$

为了得到全局特征并且维度固定的特征向量，需要引入池化层 (Pooling Layer)，可使用随时间推移最大化 (Max Over Time) 方法。

$$\left[f_{\theta}^l \right]_i = \max_x \left[f_{\theta}^{l-1} \right]_{i,t} \quad 1 \leq i \leq n_{\text{hu}}^{l-1}$$

n_{hu}^{l-1} 表示第 $l-1$ 层的隐层单元个数。

基于卷积神经网络的词性标注

HardTanh 层：

线性层通常堆叠在一起，与非线性函数交错，以提取高层非线性特征。

可选择双曲正切的“HardTanh”作为非线性函数，它的优点是计算成本略低（与精确的双曲正切相比），同时保持泛化性能不变。

$$\left[f_{\theta}^l\right]_i = \text{HardTanh}\left(\left[f_{\theta}^{l-1}\right]_i\right),$$

$$\text{HardTanh}(x) = \begin{cases} -1 & \text{if } x < -1 \\ x & \text{if } -1 \leq x \leq 1 \\ 1 & \text{if } x > 1 \end{cases}.$$

Scoring：

最后，网络最后一层 L 的输出大小等于任务的可能标签数量，每个输出都可以解释为相应标签的分数。

词性标注评价方法

采用准确率 (Accuracy) 和宏平均 F1 (Macro-F1) 两种评测指标：
宏平均 F1 需要首先计算每个词性标签的 F1 值，再计算所有词性标签 F1 值的平均值。

以名词词性标签的精确率和召回率为例：

$$P_{\text{名词}} = \frac{\text{算法输出的正确名词结果个数}}{\text{算法输出的名词结果总数}} \times 100\%$$

$$R_{\text{名词}} = \frac{\text{算法输出的正确名词结果个数}}{\text{测试集中正确名词结果总数}} \times 100\%$$

$$F1_{\text{名词}} = \frac{2 \times P_{\text{名词}} \times R_{\text{名词}}}{P_{\text{名词}} + R_{\text{名词}}}$$

词性标注评价方法

宏平均 F1 (Macro-F1) 的具体计算方法如下：

$$\text{MacroF1} = \frac{1}{n} \sum_{i \in \text{POS}} \text{F1}_i$$

其中 n 为词性标签的数量，POS 为词性标签集合。

宏平均 F1 对含有较少单词的标签类别更敏感，每个类别标签的 F1 值同等重要。

而准确率对于仅含有少量单词的标签类别的效果不敏感。

从衡量算法的能力角度，宏平均 F1 相对可以更好地反映类别不均衡情况下的算法性能。

词性标注语料库

词性标注算法的训练过程都依赖于标注语料集合。对不同算法的效果进行对比也依赖于标准测试集合。

语料库名称	单词数量	语言
英语宾州树库 (PTB)	117 万	英文
通用依存树库 (UD V2.0 CoNLL 2017)	281 万	多语言
RIT-Twitter	1 万	英文
ARK-Twitter	3 万	英文
中文宾州树库 6.0 (CTB 6.0)	78 万	中文
中文宾州树库 7.0 (CTB 7.0)	120 万	中文
中文宾州树库 8.0 (CTB 8.0)	162 万	中文
中文宾州树库 9.0 (CTB 9.0)	208 万	中文

图: 常见词性标注语料库

本章小结

第二章词汇分析

2.1 语言中的词汇

2.2 词语规范化

2.3 中文分词

2.4 词性标注